

P1272R2

Byteswapping for fun&&nuf

Published Proposal, 2019-06-14

This version:

<https://wg21.link/P1272R2>

Author:

[Isabella Muerte](#)

Audience:

LWG

Toggle Diffs:

☐ Hide deleted text

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Render:

[P1272R1](#)

Current Source:

slurps-mad-rips/papers/proposals/byteswap.bs

Implementation:

slurps-mad-rips/byteswap

Table of Contents

1 Revision History

1.1 Revision 2

1.2 Revision 1

1.3 Revision 0

2 Motivation

3 Design Considerations

3.1 Synopsis

4 Wording

4.1 Feature Testing

5 Acknowledgement

References

Informative References

§ 1. Revision History

§ 1.1. Revision 2

- Add missing remarks of undefined behavior in cases such as byteswapping an `IntegerType` such as a bitfield.
- Update feature macro value for the Cologne 2019 meeting

§ 1.2. Revision 1

- Update wording based on feedback from CWG regarding the representation of the "reversed" object.
- Update wording based on feedback from LWG regarding subclause location and grammatical changes
- Readjust wording to use [\[N4800\]](#) library wording constructs instead of [\[N4672\]](#)

§ 1.3. Revision 0

Initial release. 🎉

§ 2. Motivation

Proposal [\[P0553r2\]](#) gives several bit operations to perform on integer types, such as popcount, or bit rotation. Despite these new operations provided to C++ developers, we still cannot swap (i.e., reverse) the bytes of builtin integer types in a performant way (i.e., one instruction or less) without resorting to compiler intrinsics. Currently, most CPU architectures provide single instructions for a byte swap. For

those that don't, falling back on existing operations is more than amenable. We should, however, endeavor to standardize existing practice.

Note: The phrase one instruction or less refers to compilers inserting at most one instruction, and at the very least removing any instructions due to optimizations.

§ 3. Design Considerations

The design for the byteswap free function is quite simple. It takes any integer type and swaps its byteorder to the reverse of its current state. Additionally, it *only* takes integer types, requiring users to openly `bit_cast` their non-integers to integers in blatant heresy to the Worm-Seethe. How utterly disgraceful.

Note: It is intended that the byteswap function, despite swapping bytes, be placed into the `<bit>` header.

§ 3.1. Synopsis

The function's full specification is:

```
namespace std {  
    template <class IntegerType>  
        constexpr IntegerType byteswap (IntegerType value) noexcept;  
}
```

Where `std::is_integral_v<IntegerType>` is `true`.

§ 4. Wording

The following synopsis is to be added to subclause 25.5.2 Header `<bit>` Synopsis [**bit.syn**]:

```
// 25.5.3, bit_cast
template <typename To, typename From>
    constexpr To bit_cast (const From& from) noexcept;

// 25.5.4, byteswap
template <class IntegerType>
    constexpr IntegerType byteswap (IntegerType value) noexcept;

template <class T>
    constexpr bool ispow2(T x) noexcept;
template <class T>
    constexpr T ceil2(T x) noexcept;
...

```

The following is to be placed *before* the current subclause 25.5.4 [**bit.pow.two**] into [**bit.byteswap**]:

```
template <class IntegerType>
    constexpr IntegerType byteswap (IntegerType value) noexcept;

```

¹ *Constraints:* `std::is_integral_v<IntegerType>` is true.

² *Returns:* An object of type `IntegerType` whose representation is the reverse of the object representation of the parameter `value`.

³ *Remarks:* If there is no value of type `IntegerType` corresponding to the object representation produced, the behavior is undefined.

§ 4.1. Feature Testing

In 16.3.1 [**support.limits.general**], Table 36, add a new row below `__cpp_lib_byte` with the following content:

Macro Name	Value	Header(s)
<code>__cpp_lib_byte</code>	201603L	<cstdint>
<code>__cpp_lib_byteswap</code>	201906L	<bit>
<code>__cpp_lib_char8_t</code>	201811L	<atomic> <filesystem> <istream> <limits> <locale> <ostream> <string> <string_view>

§ 5. Acknowledgement

Thanks to Nicole Muzzuca for feedback on writing this proposal.

§ References

§ Informative References

[N4672]

Casey Carter. Editor's Report for the Ranges TS. 19 June 2017. URL: <https://wg21.link/n4672>

[N4800]

Richard Smith. Working Draft, Standard for Programming Language C++. 21 January 2019.
URL: <https://wg21.link/n4800>

[P0553r2]

Jens Maurer. Bit operations. 23 November 2017. URL: <https://wg21.link/p0553r2>